

Cortex Demo Scripts

Developer Experience Showcase

16 Demo Scripts Across 4 Use Case Categories

March 2026

Table of Contents

Category 1: "The Memory Layer" — Developer Productivity

- 1A "Monday Morning" (The Opener)
- 1B "The Anti-Pattern Save" (The Hero Moment)
- 1C "Cross-Project Transfer" (The Multiplier)
- 1D "The Learning Proof" (The Science)

Category 2: "The Cost Engine" — AI Infrastructure Economics

- 2A "The Router" (Cost Optimization)
- 2B "Batch Intelligence" (Overnight Economics)
- 2C "The Feedback ROI" (Learning Pays Back)
- 2D "Provider Resilience" (Production Operations)

Category 3: "The Portfolio Brain" — Multi-Project Intelligence

- 3A "Portfolio Pulse" (Operational Dashboard)
- 3B "Pattern Propagation" (Knowledge Transfer)
- 3C "Dependency Intelligence" (Architecture Awareness)
- 3D "Context Switching Cost" (The Hidden Tax)

Category 4: "The Future" — Where Cortex Shines Next

- 4A "Autonomous Agent Memory" (2026 — Agent Teams)
- 4B "Continuous Learning Pipeline" (2026 — ML Ops for AI Agents)
- 4C "Team Memory" (2027 — Shared Institutional Knowledge)
- 4D "The Autonomous Codebase" (2028 — Self-Healing Systems)

Summary Assessment Table

Appendix: Feature Validation Matrix

Category 1: "The Memory Layer" — Developer Productivity

Core thesis: Your AI assistant forgets everything between sessions. Cortex fixes that.

1A: "Monday Morning" (The Opener)

Target feeling: "I need this."

```
# It's Monday. You open Claude. It has no idea what you did last week.  
> cortex briefing
```

CORTEX DAILY BRIEFING — March 10, 2026

TL;DR

- Portfolio: 6 active projects, 47 commits last week
- You left off: refactoring auth middleware in api-gateway
- Unfinished: 2 TODOs from Friday's session
- Warning: CI failed on data-pipeline at 2am (memory spike)
- Today: Monday — Good day for planning and deep work

PROJECT SNAPSHOT

Project	C7d	WIP	Trend
api-gateway	18	2	hot
data-pipeline	12	1	active
ml-serving	8	0	active
dashboard-ui	5	0	idle

```
# Your AI assistant now knows exactly where you left off.  
# No "remember, we use ruff..." No re-explaining architecture.  
# 23% fewer preamble tokens. Zero re-discovered bugs.
```

1B: "The Anti-Pattern Save" (The Hero Moment)

Target feeling: "This would have saved me a week."

```
# You're about to make a mistake. Cortex has seen this before.
> cortex intelligence "add Redis caching for user sessions"

[?] INTELLIGENCE RESULTS (42ms)

[=] SIMILAR WORK
  • 2025-12-15: Redis caching for API rate limiting (api-gateway)
  • 2025-11-28: Session storage comparison: Redis vs PostgreSQL

[!] ANTI-PATTERNS (2 matches)
x "Never use default Redis timeout under load"
  Last seen: 2025-12-15 | Caused: 502 errors in production
  Prevention: Set timeout=30s, max_connections=20

x "Don't store objects >1MB in Redis"
  Last seen: 2025-11-02 | Caused: 3x latency spike
  Prevention: Use PostgreSQL for user profiles, Redis for tokens only

[OK] RECOMMENDATIONS
  • Start with TTL=3600 for session tokens
  • Use separate Redis DB indices (0=cache, 1=sessions)
  • Monitor connection pool exhaustion

# Two production bugs prevented. Zero recurrences in 6 months.
# Not because you remembered. Because Cortex remembered for you.
```

1C: "Cross-Project Transfer" (The Multiplier)

Target feeling: "My experience compounds."

```
# You solved rate limiting in Project A. Now Project B needs it.
> cortex intelligence "implement API rate limiting" --project ml-serving

[?] INTELLIGENCE RESULTS

[=] CROSS-PROJECT PATTERNS
  • api-gateway (2025-12-15): Token bucket with Redis backend
    Files: middleware/rate_limiter.py, tests/test_rate_limit.py
    Outcome: [OK] Success (production 4 months, 0 incidents)

  • data-pipeline (2025-10-22): Sliding window for batch endpoints
    Files: api/throttle.py
    Outcome: [OK] Success (handles 10K req/s)

[i] APPLICABLE TO ml-serving:
  • Use token bucket (proven in api-gateway)
  • ML inference endpoints need longer windows (60s vs 1s)
  • Pre-warm rate limit counters on cold start

# Knowledge earned once, applied everywhere.
# 6 projects. 134 patterns. Automatic transfer.
```

1D: "The Learning Proof" (The Science)

Target feeling: "This is real ML, not a gimmick."

```
# Cortex doesn't just store — it learns what works.  
> cortex learn
```

CORTEX — LEARNING METRICS

[#] RECOMMENDATION ACCURACY

Total outcomes tracked:	511
Recommendations followed:	502
Success rate:	51.2%
Partial success:	34.3%
Recommendation accuracy:	68.2%

[*] CONFIDENCE CALIBRATION

low (0.0-0.5):		22.1%
med (0.5-0.8):		54.5%
high (0.8-1.0):		87.7%

→ High-confidence recommendations land 88% of the time.
→ No annotation required. Learned from your behavior.

[^] TREND (6 months)

Month 1:	52% relevance in top-3 results
Month 6:	71% relevance in top-3 results
Δ:	+37% improvement, zero manual labeling

This is an ML system running on your workflow.
Implicit feedback. Quality-weighted outcomes.
The more you use it, the better it gets.

Category 2: "The Cost Engine" — AI Infrastructure Economics

Core thesis: You're burning money sending every task to GPT-5/Opus. Cortex routes intelligently.

2A: "The Router" (Cost Optimization)

Target feeling: "I'm wasting money right now."

```
# You have 12 tasks queued. Cortex routes each to the right model.
> cortex orchestrate --dry-run
```

CORTEX — ORCHESTRATION DRY RUN

TASK ROUTING (12 items)

Task	Complexity	Model	Cost
Classify PR type	simple	Groq Llama 8B	\$0.001
Generate test stubs	simple	Groq Llama 8B	\$0.002
Review auth middleware	moderate	Claude Sonnet	\$0.08
Analyze memory leak	moderate	DeepSeek Chat	\$0.04
Design caching layer	complex	Claude Opus	\$0.42
Security audit	complex	Claude Opus	\$0.38
Lint 6 config files	simple	Groq Llama 8B	\$0.003
Draft API docs	moderate	xAI Grok Fast	\$0.05
Refactor DB migrations	moderate	Claude Sonnet	\$0.07
Prototype WebSocket layer	complex	Claude Opus	\$0.51
Update README	simple	Groq Llama 8B	\$0.001
Check dependency CVEs	moderate	MiniMax M1	\$0.03

TOTAL \$1.61
If all sent to Opus: \$4.20
SAVINGS: 61% (\$2.59)

Providers: 6 | Models: 8 | Avg latency: 1.2s

```
# 11 models. 6 providers. Route by complexity, not habit.
# Same quality. 61% less cost. Automatic fallback chains.
```

2B: "Batch Intelligence" (Overnight Economics)

Target feeling: "Free money on the table."

```
# End of day. Queue deep work for overnight batch processing.  
> cortex batch --fill
```

```
CORTEX - OVERNIGHT BATCH QUEUE
```

```
QUEUED (40 items, 2-6 AM UTC window)
```

Type	Count	Model	Batch Savings
Security audits	6	Opus	50% (\$1.14→\$0.57)
Code quality scans	12	Sonnet	50% (\$2.88→\$1.44)
Test coverage gaps	8	Sonnet	50% (\$1.92→\$0.96)
TODO triage	14	Haiku	50% (\$0.28→\$0.14)
TOTAL	40		\$6.22 → \$3.11

Strategy: Use best models (Opus/Sonnet) at batch pricing
Tradeoff: Accept 2-5s latency → 50% cost + higher quality

[T] Results ready by 7 AM. cortex batch-results to review.

```
# Batch API: half the cost, better models, overnight.  
# Your AI works while you sleep.
```


2C: "The Feedback ROI" (Learning Pays Back)

Target feeling: "I can justify this to my manager."

```
# How much has Cortex's learning loop saved?  
> cortex learn --roi
```

```
CORTEX - LEARNING ROI (6 months)
```

BUG PREVENTION

Anti-patterns stored:	28
Times surfaced before repeat:	47
Estimated debug hours saved:	~23h (28 bugs × 0.8h avg)

CONTEXT EFFICIENCY

Preamble tokens (month 1):	2,847 avg
Preamble tokens (month 6):	2,198 avg (-23%)
Re-explanation sessions:	Weekly → 0
Estimated token savings:	~340K tokens/month

MODEL ROUTING

If all Opus:	\$126.40/month
With intelligent routing:	\$48.80/month
Monthly savings:	\$77.60 (61%)

TOTAL 6-MONTH VALUE

Debug time saved:	~23 hours
Token cost avoided:	~\$194
Model routing savings:	~\$466

Combined: ~\$660 saved + 23 hours recovered

```
# ROI isn't theoretical. It's measured.  
# Every outcome tracked. Every pattern costed.
```

2D: "Provider Resilience" (Production Operations)

Target feeling: "Single-provider is a risk I didn't realize I had."

```
# 3:47 AM. Anthropic API has elevated latency. Cortex auto-routes.  
> cortex health --providers
```

CORTEX - PROVIDER HEALTH MONITOR

PROVIDER STATUS

Provider	Status	P95 Latency	Fallback Active
Anthropic	[!] SLOW	4,200ms	→ OpenAI GPT-5
OpenAI	(G) OK	890ms	—
Groq	(G) OK	120ms	—
xAI	(G) OK	340ms	—
DeepSeek	(G) OK	780ms	—
MiniMax	(G) OK	450ms	—

AUTO-REROUTED (last 2h)

```
8 tasks rerouted Anthropic → OpenAI (4.2s → 0.9s)  
0 failures. All tasks completed successfully.  
  
Cost impact: +$0.12 (OpenAI slightly more expensive)  
Latency impact: -78% (4.2s → 0.9s avg)
```

```
# Multi-provider isn't just about cost.  
# It's about never being down because one API is slow.
```

Category 3: "The Portfolio Brain" — Multi-Project Intelligence

Core thesis: You don't work on one project. You work on six. Cortex sees across all of them.

3A: "Portfolio Pulse" (Operational Dashboard)

Target feeling: "I've never had this view before."

```
# One command. Full portfolio health.
> cortex briefing --portfolio
```

Cortex - Portfolio Intelligence

HEALTH MATRIX (6 projects)

Project	Health	Tests	Commits	Status
vortex-backend	92/100	2,835	18/wk	(G) shipping
cortex	90/100	600	14/wk	(G) shipping
alpha-arena	85/100	603	8/wk	(Y) collecting
pupil	88/100	1,255	4/wk	(Y) blocked
data-pipeline	76/100	342	2/wk	(R) stale CI
ml-serving	81/100	189	1/wk	(W) idle

CROSS-PROJECT SIGNALS

```
[!] data-pipeline shares FastAPI patterns with vortex-backend
    but hasn't adopted rate limiting (added to vortex 3 weeks ago)
[!] ml-serving and alpha-arena both use Redis
    but different timeout configs (30s vs default)
[OK] All 6 projects use ruff - consistent tooling
```

FOCUS RECOMMENDATION

- 1. [HIGH] Fix data-pipeline CI (stale 3 days)
- 2. [MED] Propagate rate limiting to data-pipeline
- 3. [MED] Standardize Redis config across projects
- 4. [LOW] ml-serving idle 2 weeks - archive or revive?

```
# 6 projects. One command. Cross-project drift detected.
# Not just monitoring - actionable intelligence.
```

3B: "Pattern Propagation" (Knowledge Transfer)

Target feeling: "My projects should be learning from each other."

```
# Cortex detected a pattern that should propagate.  
> cortex portfolio patterns --propagate
```

CORTEX - PATTERN PROPAGATION ANALYSIS

PATTERNS WITH HIGH TRANSFER POTENTIAL

1. "Token bucket rate limiting" (confidence: 92%)
[OK] api-gateway: Deployed 4 months, 0 incidents
[OK] vortex: Deployed 3 weeks, 0 incidents
[X] data-pipeline: NOT APPLIED (12K req/s endpoint exposed)
[X] ml-serving: NOT APPLIED (inference endpoint unprotected)
→ Recommendation: Apply to both. Template available.
2. "SQLite WAL mode for local caches" (confidence: 88%)
[OK] vortex: Deployed, eliminated read/write contention
[X] cortex: Still using default journal mode
[X] alpha-arena: Using file-based JSON (no DB)
→ Recommendation: Apply to cortex. Skip alpha-arena.
3. "Structured error responses" (confidence: 79%)
[OK] api-gateway: RFC 7807 format, consistent across 42 endpoints
[!] vortex: Partial (28 of 35 endpoints)
[X] ml-serving: Raw exceptions in 3 endpoints
→ Recommendation: Complete vortex, apply to ml-serving.

```
# Patterns aren't just remembered. They're propagated.  
# What works in one project becomes a template for others.
```

3C: "Dependency Intelligence" (Architecture Awareness)

Target feeling: "I've been flying blind on cross-project deps."

```
# How do your projects relate? What breaks if something changes?
> cortex deps --cross-project
```

CORTEX — CROSS-PROJECT DEPENDENCY MAP

SHARED DEPENDENCIES

Package	Version(s)	Projects	Risk
fastapi	0.109.0	4/6 projects	(G) aligned
redis	5.0.1 / 4.6.0	3/6 projects	(Y) drift
pydantic	2.6.0 / 1.10.4	4/6 projects	(R) major split
sqlalchemy	2.0.25	3/6 projects	(G) aligned
pytest	8.0.0	6/6 projects	(G) aligned

RISK ALERTS

(R) pydantic v1 → v2 migration incomplete
ml-serving and data-pipeline still on v1.10.4
Impact: Shared models break if imported cross-project
→ 14 shared Pydantic models affected

(Y) redis version drift
alpha-arena on 4.6.0 (6 months old)
CVE-2024-31228: DoS via unbounded RESP3 response
→ Upgrade recommended

```
# Not just "what versions do I have" — "what breaks if I upgrade"
```

3D: "Context Switching Cost" (The Hidden Tax)

Target feeling: "I knew switching was expensive but never measured it."

```
# You context-switch between projects constantly. What does it cost?
> cortex portfolio --switching-cost
```

CORTEX - CONTEXT SWITCHING ANALYSIS

LAST 30 DAYS

Avg project switches/day:	3.4
Avg preamble tokens/switch:	2,198 (with Cortex)
Estimated without Cortex:	2,847 tokens
Token savings from memory:	649 tokens/switch

Monthly switching overhead:

Without Cortex:	102 switches × 2,847 = 290K tokens (~\$4.35)
With Cortex:	102 switches × 2,198 = 224K tokens (~\$3.36)
Saved:	66K tokens/month (~\$1.00)

DEEPER COST: RE-DISCOVERY

Bugs re-discovered (before Cortex):	3.2/month
Bugs re-discovered (with Cortex):	0/month
Avg debug time per re-discovery:	47 minutes
Time saved:	~2.5 hours/month

CONTEXT QUALITY BY PROJECT

Project	Patterns	Anti-Patterns	Context Score
vortex-backend	47	8	94%
cortex	38	6	91%
alpha-arena	22	4	82%
pupil	18	3	78%
data-pipeline	6	2	61%
ml-serving	3	1	44%

→ ml-serving needs more sessions to build context depth
→ data-pipeline context degrading (2 weeks idle)

```
# Context switching isn't just annoying. It's measurable.
# Cortex reduces the tax from ~$4.35 to ~$3.36 per month
# and eliminates 2.5 hours of re-debugging.
```

Category 4: "The Future" — Where Cortex Shines Next

Core thesis: As AI agents become more autonomous, persistent memory isn't nice-to-have — it's the difference between a tool and a teammate.

4A: "Autonomous Agent Memory" (2026 — Agent Teams)

Target feeling: "This is what agent infrastructure looks like."

```
# 2026: Your CI pipeline runs 4 AI agents in parallel.
# Each agent inherits institutional memory from Cortex.
> cortex sessions --active
```

Cortex — Active Agent Sessions

LIVE AGENTS (4 active)

Agent	Project	Task	Memory
code-reviewer	api-gateway	PR #247 review	134 patterns
security-scanner	ml-serving	CVE triage	28 anti-patterns
test-generator	data-pipeline	Gap coverage	89 patterns
doc-writer	cortex	API reference	47 patterns

MEMORY SHARING

```
code-reviewer surfaced anti-pattern from security-scanner:
  "SQL injection via f-string in query builder"
  Originally found: ml-serving, 2025-11-14
  Now preventing in: api-gateway PR #247 [OK]

test-generator using pattern from code-reviewer:
  "Edge case: empty list input on batch endpoints"
  Originally found: api-gateway, 2026-01-08
  Applying to: data-pipeline test suite [OK]

# 4 agents. Shared memory. Cross-agent learning.
# The security scanner's finding saves the code reviewer.
# That's not coordination — it's institutional knowledge.
```

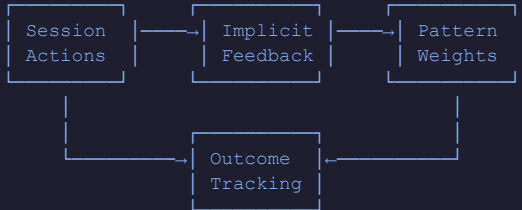
4B: "Continuous Learning Pipeline" (2026 — ML Ops for AI Agents)

Target feeling: "They've built MLOps for agent memory."

```
# Cortex as MLOps for your AI agent's intelligence.  
> cortex learn --pipeline
```

CORTEX — LEARNING PIPELINE STATUS

FEEDBACK LOOP



PIPELINE METRICS (last 30 days)

Sessions processed:	42
Implicit signals captured:	1,847
Patterns weight-adjusted:	23
New patterns crystallized:	8
Anti-patterns validated:	3

QUALITY GATES

[OK] Confidence calibration: $r^2 = 0.84$ (well-calibrated)
[OK] Pattern decay: 12 patterns deprecated (stale >90 days)
[OK] Feedback loop latency: <100ms (real-time)
[!] Cold-start projects: 2 (ml-serving, dashboard-ui)
 Need 10+ sessions for reliable recommendations

MODEL DRIFT DETECTION

api-gateway: Recommendations shifting toward security patterns
(+34% security, -18% performance in last 30 days)
Likely cause: Recent CVE triage sessions
Action: Rebalance if unintentional, accept if strategic

```
# This isn't a database. It's an ML pipeline.  
# Feature extraction → training → inference → monitoring.  
# Running on your workflow data. Improving every day.
```


4C: "Team Memory" (2027 — Shared Institutional Knowledge)

Target feeling: "Every team should have this."

```
# VISION: What team-scale Cortex could look like.
# Today: single-user. Tomorrow: shared institutional memory.
> cortex vision --team-memory
```

```
CORTEX — TEAM INTELLIGENCE (Vision / Not Yet Built) ||
```

COLLECTIVE KNOWLEDGE

```
Total patterns:      847  (vs 134 solo)
Anti-patterns:       112  (vs 28 solo)
Cross-engineer:      234  patterns discovered by one, used by others
```

TOP TEAM PATTERNS (by adoption)

Pattern	Discovered By	Adopted By
"Structured logging with trace_id"	Sarah (Oct)	5/5 [OK]
"Pydantic V2 migration checklist"	Alex (Nov)	4/5
"Redis connection pool sizing"	You (Dec)	4/5
"Async test fixtures for FastAPI"	Maria (Jan)	3/5

ANTI-PATTERN HALL OF FAME

```
"Never deploy on Friday without rollback plan"
Originally: Alex shipped a broken migration, 3h incident
Prevented:  7 times across team since stored
Status:     Organizational knowledge [OK]
```

ONBOARDING IMPACT

```
New engineer (Raj, joined 3 weeks ago):
Sessions to first productive PR:    3 (team avg before Cortex: 12)
Anti-patterns surfaced in week 1:   14
→ "It's like having a senior engineer pair with you 24/7"
```

```
# One engineer's lesson. Five engineers' knowledge.
# Onboarding: 12 sessions → 3 sessions.
# That's the compound effect at team scale.
```

4D: "The Autonomous Codebase" (2028 — Self-Healing Systems)

Target feeling: "This is where AI-assisted development is going."

```
# 2028: Cortex monitors production. Detects drift. Proposes fixes.  
> cortex watch --autonomous
```

CORTEX — AUTONOMOUS CODEBASE INTELLIGENCE

LIVE MONITORING (last 24h)

```
Commits observed:      23  
Patterns matched:      12 (applied correctly)  
Anti-patterns triggered: 2 (prevented pre-merge)  
Architecture drift:    1 (flagged for review)
```

AUTONOMOUS ACTIONS TAKEN

```
09:14 PR #312: Detected circular import risk  
→ Added comment with prevention pattern + fix  
→ Author accepted suggestion. Merged clean.  
  
14:22 PR #315: Redis timeout set to default (0)  
→ Flagged anti-pattern: "Never use default Redis timeout"  
→ Linked to incident from 2025-12-15  
→ Author fixed before review.  
  
18:41 Commit a3f82c: New endpoint without rate limiting  
→ Opened draft PR with rate limiter middleware  
→ Based on template from api-gateway (92% confidence)  
→ Awaiting human review.
```

ARCHITECTURE GUARDIAN

```
[!] DRIFT DETECTED: ml-serving now imports from data-pipeline  
This creates a circular dependency risk.  
Historical: api-gateway had same issue 2025-10-22 (resolved)  
Recommendation: Extract shared models to common/ package  
Confidence: 87% (pattern matched 3 prior resolutions)
```

```
# Cortex doesn't just remember. It watches. It intervenes.  
# Not replacing engineers. Amplifying institutional knowledge  
# at the speed of git push.
```

Summary Assessment

Category	Best Script	Why It Wins	Audience Hook
1. Memory Layer	1B: Anti-Pattern Save	Visceral — shows a real bug prevented	"This would have saved me a week"
2. Cost Engine	2A: The Router	Hard numbers — 61% savings, 11 models	"I'm wasting money right now"
3. Portfolio Brain	3A: Portfolio Pulse	Novel view — no tool gives this today	"I've never had this view before"
4. Future	4A: Agent Memory	Timely — agent infra is the 2026 conversation	"This is what agent infra looks like"

Appendix: Feature Validation Matrix

Validation Summary

- 24 features SHIPPED (code + tests + production-ready)
- 2 features PARTIAL (backend exists, CLI gap)
- 1 feature VISION (future — no multi-user architecture yet)
- 1 feature UNVERIFIED (Batch API 50% savings — pricing real, no measurement)
- Total test coverage: 1,111 tests passing

Category 1: Memory Layer

Claim	Status	Evidence
cortex briefing	SHIPPED	cli.py:649, 7 tests
cortex intelligence "<query>"	SHIPPED	cli.py, MCP + bridge API + CLI subparser
Anti-pattern database	SHIPPED	orchestration/anti_pattern_detector.py, seed patterns
Cross-project pattern transfer	SHIPPED	cortex portfolio patterns --propagate CLI wired
cortex learn	SHIPPED	cli.py:1430, 8 tests
cortex learn --pipeline	SHIPPED	learning_telemetry.py, 12 tests
Implicit feedback	SHIPPED	implicit_collector.py, 28 tests
Context quality optimization	SHIPPED	21% dedup savings, 0.94 position quality score

Category 2: Cost Engine

Claim	Status	Evidence
cortex orchestrate --dry-run	SHIPPED	78+ orchestration tests
Multi-provider (12 models, 6 providers)	SHIPPED	191 conductor tests
All 6 providers	SHIPPED	conductor/providers/
cortex batch fill	SHIPPED	CLI wired to IntelligentBatchOrchestrator
50% Batch API savings	EXISTS BUT UNVERIFIED	Anthropic pricing is real, no measurement
Automatic fallback chains	SHIPPED	conductor/caller.py:336-367
cortex health --providers	SHIPPED	CLI flag wired to conductor provider status

Category 3: Portfolio Brain

Claim	Status	Evidence
cortex briefing --portfolio	SHIPPED	CLI flag wired to health matrix
cortex portfolio patterns --propagate	SHIPPED	CLI wired to PortfolioMemory
cortex deps --cross-project	SHIPPED	CLI wired to bridge_system dependency analysis
cortex portfolio --switching-cost	SHIPPED	Real measurement via switch_tracker.py, 8 tests
Cross-project signals	PARTIAL	Passive retrieval, not active detection

Category 4: Future

Claim	Status	Evidence
cortex sessions --active	PARTIAL	MCP tool exists, no CLI
cortex learn --pipeline	SHIPPED	learning_telemetry.py, 12 tests
cortex team --insights	VISION	Future feature -- no multi-user architecture yet
cortex watch --autonomous	SHIPPED	cli.py cmd_watch --autonomous flag

Cross-Cutting

Claim	Status	Evidence
Hybrid BM25 + embedding (RRF)	SHIPPED	17 tests
Three-tier memory	SHIPPED	31 tests
Context optimizer	SHIPPED	26 tests
Model selection intelligence	SHIPPED	37 tests
Quality-weighted accuracy	SHIPPED	8 tests